Memory Allocation

Implement a **First Fit** memory allocation scheme using a simple approach in C. You can follow these instructions to write your own version of the program:

1. Set Up the Environment:
- Include necessary headers: #include <stdio.h>.
- Define any necessary constants, such as the maximum number of blocks/files (#define MAX 25).

2. Declare Variables:
- Create arrays to hold the sizes of blocks and files (int blockSize[MAX], fileSize[MAX];).
- Create arrays to track the allocation status (int blockAllocated[MAX] = {0}; for whether a block is allocated, int fileBlockIndex[MAX]; for which block is allocated to each file).
- Declare other necessary variables, like `int numBlocks, numFiles;` to store the number of blocks and files.

3. Input Number of Blocks and Files:
- Use `scanf` to read the number of blocks and files from the user.

```
printf("Enter the number of blocks: ");
scanf("%d", &numBlocks);
printf("Enter the number of files: ");
scanf("%d", &numFiles);
```

4. Input Block Sizes:
- Use a loop to read each block's size.

```
printf("Enter the size of each block:\n");
for (int i = 0; i < numBlocks; i++) {
    printf("Block %d: ", i + 1);
    scanf("%d", &blockSize[i]);
}
```

5. Input File Sizes:
- Use a loop to read each file's size.

```
printf("Enter the size of each file:\n");
for (int i = 0; i < numFiles; i++) {
    printf("File %d: ", i + 1);
    scanf("%d", &fileSize[i]);
}
```

6. Implement First Fit Allocation:
- For each file, find the first block that is large enough to store it and is not already allocated.
- If such a block is found, mark it as allocated.

This part of code is your task to do

7. Display the Allocation Results:
- Print a table showing each file, the block it is allocated to, the block size, and the fragmentation (if applicable).
- Ensure that each file has been properly matched, and handle cases when a file cannot be allocated.

8. Output Example Code:

```
` printf("\nFile No\tFile Size\tBlock No\tBlock Size\n");
for (int i = 0; i < numFiles; i++) {
    if (blockAllocated[fileBlockIndex[i]]) {
        printf("%d\t\t%d\t\t%d\t\t%d\n", i + 1, fileSize[i], fileBlockIndex[i] + 1,
blockSize[fileBlockIndex[i]]);
    } else {
        printf("%d\t\t%d\t\tNot Allocated\n", i + 1, fileSize[i]);
    }
}
`
```

Expected Output:

```
Hinds-MacBook-Pro:Lab#2 hindalsharif$ cc lab2.c
Hinds-MacBook-Pro:Lab#2 hindalsharif$ ./a.out
First Fit Memory Allocation
Enter the number of blocks: 3
Enter the number of files: 2
Enter the size of each block:
Block 1: 20
Block 2: 15
Block 3: 40
Enter the size of each file:
File 1: 30
File 2: 10

File No File Size       Block No        Block Size
1               30              3               40
2               10              1               20
```

Tips:
- Include error checking for invalid inputs.
- Add comments to your code for better readability.
- Think about what should happen if a file doesn't fit into any block, and how they should handle fragmentation.

Submission guidelines:

1. File Format: Submit your work as .pdf
2. Code submission: Submit your codes as source code (.c)
3. Screenshots: Include at necessary screenshot of your exercises output.
4. Deadline: Submit your file before the due date announced in blackboard. Late submissions would lead to lose marks
5. Integrity in your work is fundamental to your success and character. Always strive to do your own best work, cite sources properly, and be honest about your efforts.